

Computer Networks

Week 7 — Detailed Study Notes

The Network Layer · IPv4 Addressing, Prefixes, CIDR & Netmasks · Inside a Router ·
Route Lookup & Longest Prefix Matching · IPv4 Datagram Format · ICMP, ping &
traceroute

IIT Madras BS — Prof. Ayon Chakraborty

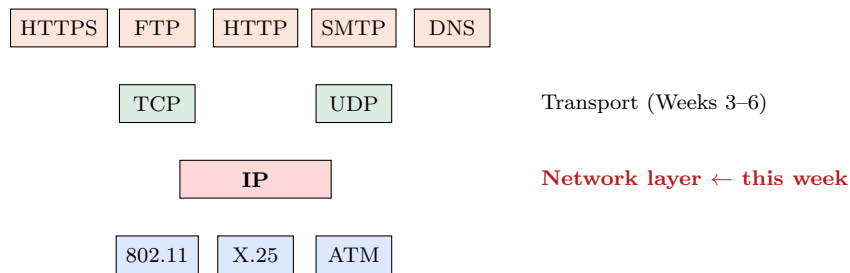
How to read these notes. Every slide from the three Week-7 lectures appears here, in slide order, grouped by module (Module 7. n = Lecture n), with the professor's figures embedded. Box types as before. The end has a formula sheet, numerical problems with full solutions, and MCQs with explanations.

Contents

1	Module 7.1 — The Network Layer and IPv4 Addressing	2
1.1	Where we are	2
1.2	The Internet needs addresses	2
1.3	IPv4 addresses	3
1.4	Grouping IP addresses by prefixes	3
1.5	IP addresses use hierarchy to scale routing	4
1.6	Classless IPv4 addressing (CIDR)	5
1.7	Netmask (subnet mask)	6
1.7.1	Detecting addresses on the same network	6
1.8	Finding your own IP address	7
2	Module 7.2 — Inside a Router: Route Lookup and Longest Prefix Matching	7
2.1	Kinds of routers	7
2.2	What's inside a router?	8
2.3	Route lookups	8
2.4	Review, and IP block reallocation	9
2.5	A closer look at the forwarding table	10
2.6	Longest Prefix Matching (LPM)	11
3	Module 7.3 — The IPv4 Datagram and ICMP	12
3.1	IPv4 datagram format	12
3.2	Various support protocols	13
3.3	Internet Control Message Protocol (ICMP)	14
3.4	Ping	14
3.5	Traceroute	15
3.6	Summary of ICMP	16
4	Formula Sheet	17
5	Numerical Problems with Worked Solutions	17
6	Multiple-Choice Questions with Explanations	19

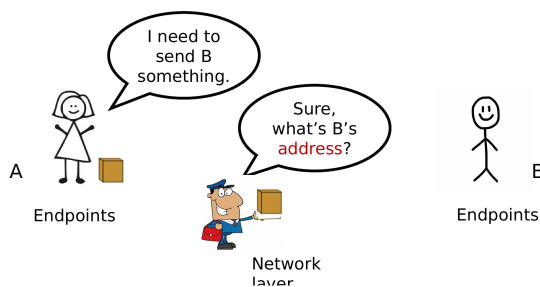
1 Module 7.1 — The Network Layer and IPv4 Addressing

1.1 Where we are



From the lecture slides: W7 L1, slides 3 and 5 — “what’s B’s address?”; the Internet needs addresses

Addresses in the Internet



The Internet needs addresses

- Addresses are a precursor to communication.
 - Allow endpoints to **locate each other**
- Internet addresses are neither endpoint names nor identify them
- Addresses help routers determine how to move a packet
 - Like **street address** for the postal system
- Network layer addresses are **designed** to help routers perform the forwarding and routing functions **efficiently**
 - Specifically, we'll look at **Internet Protocol (IP)** addresses.
 - Most popular: IP version 4 or IPv4. (later: IPv6)

1.2 The Internet needs addresses

What the slide says (W7 L1, slide 5)

Addresses are a precursor to communication: they allow endpoints to locate each other. Internet addresses are **neither endpoint names nor identify them**. Addresses help routers determine how to move a packet — like a street address for the postal system. Network layer addresses are **designed** to help routers perform the forwarding and routing functions **efficiently**. Specifically: Internet Protocol (IP) addresses. Most popular: IP version 4 (IPv4); later IPv6.

Intuition: Name vs address

“Deon” is a name — it stays with you. “Flat 4B, HSR Layout, Bengaluru 560102” is an address — it tells the postman *where* to go, and it changes when you move. IP addresses are addresses in this sense. Names live elsewhere (DNS: `iitm.ac.in`), and identities elsewhere again (MAC addresses name an interface’s hardware, certificates identify servers).

1.3 IPv4 addresses

From the lecture slides: W7 L1, slides 6–7 — 32 bits, identifies an interface (point of attachment); dotted quad; grouping by prefix

IPv4 Addresses



- 32 bits long
- Identifier for a network **interface**
- An IP address corresponds to the **point of attachment** of an endpoint to the network.
- An IP address is **NOT an identifier** for the endpoint
 - Changes when endpoint moves
- **Dotted quad notation**: each byte is written in decimal in MSB order, separated by dots. Example:

```
10000000 11000011 00000001
01010000 195 . 1 . 80
```

Grouping IP addresses by prefixes

- IP addresses can be grouped based on a **shared prefix of a specified length**
- Example: consider two IP addresses:
 - 128.95.1.80 and 128.95.1.4
 - The addresses share a prefix of (bit) length 24: 128.95.1
 - The addresses have different suffixes of (bit) length 8
- IP addresses: prefix corresponds to the **network component** and the suffix to an **endpoint (host) component** of the address

What the slide says (W7 L1, slide 6)

32 bits long. An identifier for a network **interface**. An IP address corresponds to the **point of attachment** of an endpoint to the network. An IP address is **NOT** an identifier for the endpoint — it changes when the endpoint moves. **Dotted quad notation**: each byte is written in decimal in MSB order, separated by dots. Example: 10000000 11000011 00000001 01010000 = 128.195.1.80.

Formula: Dotted quad $\leftrightarrow$ binary

Each of the four bytes is an 8-bit number 0–255:

$$a.b.c.d \leftrightarrow a \cdot 2^{24} + b \cdot 2^{16} + c \cdot 2^8 + d$$

Byte to binary: write it as a sum of 128, 64, 32, 16, 8, 4, 2, 1. E.g. 195 = 128 + 64 + 2 + 1 = 11000011, 80 = 64 + 16 = 01010000. Total address space: $2^{32} \approx 4.29 \times 10^9$ addresses.

Intuition: Interface, not host

Your laptop has a WiFi interface and maybe an Ethernet interface: two IP addresses. A router has one address per port (it sits on several networks). Carry your laptop from home to IITM and it gets a new address — same machine, new point of attachment.

Beyond the slides: See it yourself

`ip addr` on Linux (or `ifconfig -a`) lists each interface with its address in CIDR form, e.g. `inet 192.168.1.23/24` — address and prefix length together. You'll also see `lo` (loopback, 127.0.0.1/8) and each interface's MAC address (`link/ether`).

1.4 Grouping IP addresses by prefixes

What the slide says (W7 L1, slide 7)

IP addresses can be grouped based on a shared **prefix** of a specified length. Example: 128.95.1.80 and 128.95.1.4 share a prefix of (bit) length 24: 128.95.1; they have different suffixes of (bit) length 8. In IP addresses, the **prefix** corresponds to the **network** component and the **suffix** to an **endpoint (host)** component of the address.

$$\begin{array}{rcl}
 128.95.1.80 & = & \underbrace{10000000 \ 01011111 \ 00000001}_{\text{shared 24-bit prefix: network}} \underbrace{01010000}_{\text{8-bit suffix: host}} \\
 128.95.1.4 & = & \underbrace{10000000 \ 01011111 \ 00000001}_{\text{shared 24-bit prefix: network}} \underbrace{00000100}_{\text{8-bit suffix: host}}
 \end{array}$$

Common pitfall: “Share a prefix of length 24”

80 = 01010000 and 4 = 00000100 agree in their first bit too, so the two addresses actually share a 25-bit prefix. “Sharing a /24” means “agreeing on at least the first 24 bits”; which prefix length matters depends on how the network was configured, not on the addresses alone.

1.5 IP addresses use hierarchy to scale routing

From the lecture slides: W7 L1, slides 8–9 — prefixes are like PIN codes; how do we find the prefix? classful vs classless

IP addresses use **hierarchy** to scale routing

- IP addresses of endpoint interfaces in a network (e.g., each department) **share a prefix** of some length
- Each interface/endpoint has a **different suffix**, and hence a different 32-bit IP address
- Using prefixes reduces the amount of information needed to forward packets over the Internet
- IP prefixes are like **pin codes**: routers don't need to store info for each endpoint, just each prefix
- Prefixes also allow IP addresses to be **delegated** from one network to another (more on this later)



IP addresses use **hierarchy** to scale routing

- Postal envelopes should show clearly delineated pin codes.
- Q: How to identify the prefix from a 32-bit IP address?
 - Two methods:
- Old: Classful addressing. IP address itself is formatted to denote the IP prefix
- New: Classless addressing (also called classless inter-domain routing, or **CIDR**).
 - Each router independently identifies prefix from IP



What the slide says (W7 L1, slide 8)

IP addresses of endpoint interfaces in a network (e.g. each department) share a prefix of some length; each interface has a different suffix, hence a different 32-bit address. Using prefixes **reduces the amount of information needed to forward packets** over the Internet. IP prefixes are like **PIN codes**: routers don't need to store info for each endpoint, just each prefix. Prefixes also allow IP addresses to be **delegated** from one network to another.

Intuition: PIN codes

A letter from Delhi to “Bangalore 560036” is sorted in Delhi by PIN only; nobody in Delhi knows your street. Only the local post office looks at the house number. Likewise, a router in Mumbai only needs “prefix 103.21.0.0/16 → port 5”, not millions of individual host entries. The forwarding table grows with the number of *networks*, not the number of *hosts*.

What the slide says (W7 L1, slide 9)

Postal envelopes should show clearly delineated PIN codes. Q: how to identify the prefix from a 32-bit IP address? Two methods. **Old: classful addressing** — the IP address itself is formatted to denote the IP prefix. **New: classless addressing** (classless inter-domain routing, **CIDR**) — each router independently identifies the prefix from the IP.

Beyond the slides: Classful addressing (historical)

Class	Leading bits	First byte	Prefix length	Hosts per network
A	0	0–127	/8	2^{24} (≈ 16.7 M)
B	10	128–191	/16	2^{16} (65,536)
C	110	192–223	/24	2^8 (256)
D	1110	224–239	multicast	—

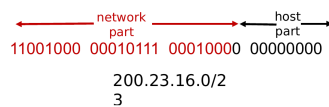
Problem: only three sizes. An organisation needing 2,000 addresses got a whole class B (65,536), wasting 97%. The address space ran out fast, and routing tables exploded with class C entries. CIDR (1993) fixed both.

1.6 Classless IPv4 addressing (CIDR)

From the lecture slides: W7 L1, slides 11–12 — a.b.c.d/x notation; an ISP partitions 200.8.0.0/16 and hands out 200.8.4.128/26

Classless IPv4 addressing

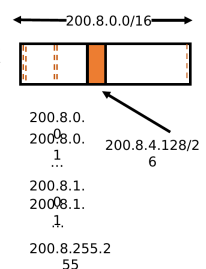
- Also called classless inter-domain routing (CIDR)
- Key idea: Network component of the address (ie: prefix) can have **any length** (usually from 8–32)
- Address format: **a.b.c.d/x**, where x is the prefix length
 - Customary to use 0s for all suffix bits



CIDR

<https://www.internetsociety.org/resources/deploy360/2015/short-guide-ip-addressing/>

- An ISP can obtain a block of addresses and partition this further to its customers
- Say an ISP has 200.8.0.0/16 address (65K addresses).
- The ISP has customer who needs only 64 addresses starting from 200.8.4.128
- Then that block can be specified as 200.8.4.128/26
- 200.8.4.128/26 is “inside” 200.8.0.0/16



What the slide says (W7 L1, slide 11)

Also called classless inter-domain routing (CIDR). Key idea: the network component of the address (the prefix) can have **any length** (usually from 8 to 32). Address format: **a.b.c.d/x**, where x is the prefix length. Customary to use 0s for all suffix bits. Example: 11001000 00010111 00010000 00000000 = **200.23.16.0/23** (23 network bits, 9 host bits).

Formula: Size of a CIDR block

A block *a.b.c.d/x* has $32 - x$ host bits, so it contains

$$N = 2^{32-x} \quad [\text{addresses}]$$

It runs from the network address (all host bits 0) to the broadcast address (all host bits 1). E.g. /24: 256; /23: 512; /20: 4096; /26: 64; /16: 65,536.

Beyond the slides: Usable host addresses

In a real subnet the all-zeros (network) and all-ones (broadcast) addresses are reserved, so usable hosts = $2^{32-x} - 2$ (e.g. /24 $\rightarrow$ 254). The slides count all addresses in a block.

What the slide says (W7 L1, slide 12 — CIDR delegation)

An ISP can obtain a block of addresses and partition it further for its customers. Say an ISP has 200.8.0.0/16 (65K addresses). A customer needs only 64 addresses starting from 200.8.4.128.

That block can be specified as **200.8.4.128/26**. 200.8.4.128/26 is “inside” 200.8.0.0/16.

Formula: Is block B inside block A?

$B = b/y$ is inside $A = a/x$ iff $y \geq x$ and b and a agree on the first x bits. For 200.8.4.128/26 in 200.8.0.0/16: $26 \geq 16$ and the first 16 bits are both 200.8 $\Rightarrow$ yes. Range of the /26: host bits = 6 $\Rightarrow$ 64 addresses: 200.8.4.128 – 200.8.4.191.

Common pitfall: Alignment

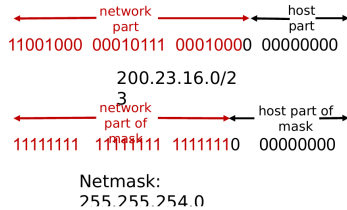
A /26 must start at a multiple of 64 in its last byte (0, 64, 128, 192). 200.8.4.128 is aligned; 200.8.4.100/26 would not be a valid block start (its network address would be 200.8.4.64).

1.7 Netmask (subnet mask)

From the lecture slides: W7 L1, slides 13–14 — the netmask; detecting whether two addresses are on the same network

Netmask (or subnet mask)

- An alternative to denote the IP prefix length of an organization
- 32 bits: a **1-bit** denotes a prefix bit position. 0 denotes host bit.



Detecting addresses from same network

- Given IP addresses A and B, and netmask M.
 1. Compute logical AND (A & M).
 2. Compute logical AND (B & M).
 3. If (A & M) == (B & M) then A and B are on the same network.
- Ex: A = 165.230.82.52, B = 165.230.24.93, M = 255.255.128.0
- A and B are in the same network according to the netmask
- A & M == B & M == 165.230.0.0

What the slide says (W7 L1, slide 13)

An alternative way to denote the IP prefix length of an organisation. 32 bits: a **1-bit** denotes a prefix bit position, a **0** denotes a host bit. For 200.23.16.0/23: 11111111 11111111 11111110 00000000 = **255.255.254.0**.

Formula: Prefix length $\rightarrow$ netmask

The netmask for / x is x ones followed by $32 - x$ zeros. The partial byte uses these values:

ones in the byte	0	1	2	3	4	5	6	7	8
byte value	0	128	192	224	240	248	252	254	255

/8 = 255.0.0.0, /16 = 255.255.0.0, /19 = 255.255.224.0, /20 = 255.255.240.0, /23 = 255.255.254.0, /24 = 255.255.255.0, /26 = 255.255.255.192.

1.7.1 Detecting addresses on the same network

What the slide says (W7 L1, slide 14)

Given IP addresses A and B and netmask M: (1) compute A & M; (2) compute B & M; (3) if (A & M) == (B & M), A and B are on the same network. Example: A = 165.230.82.52, B = 165.230.24.93, M = 255.255.128.0. A & M == B & M == 165.230.0.0 $\Rightarrow$ same network.

	third byte	result
A = 165.230.82.52	82 = 0101 0010	
M = 255.255.128.0	128 = 1000 0000	
A & M	0000 0000	165.230.0.0
B = 165.230.24.93	24 = 0001 1000	
B & M	0000 0000	165.230.0.0

Intuition: Why hosts need this

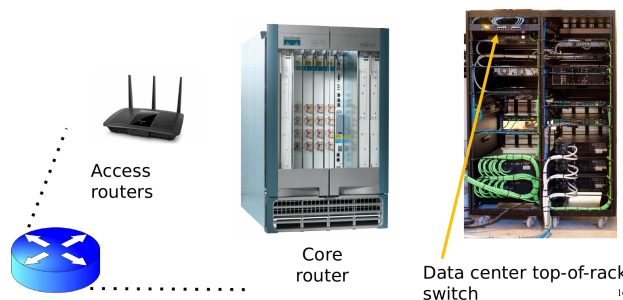
Before sending, your laptop checks: “is the destination on my own network?” If yes, deliver directly on the local link (finding its MAC address with ARP). If no, send the packet to the **default gateway** (router). The AND with the netmask is exactly that test. Only bytes where the mask is neither 255 nor 0 need real work: 255 keeps the byte, 0 zeroes it.

1.8 Finding your own IP address

From the lecture slides: W7 L1, slides 15–16 — `ifconfig` / `ip addr`; routers are next: access, core, data-centre top-of-rack

Finding your own IP address(es) Next we'll talk about routers

- The old way (still works today on Mac and Linux):
 - `ifconfig -a`
- The new way using “iproute2” tools on Linux:
 - `ip link`
 - `ip addr`
- What else do you see in these outputs?



What the slide says (W7 L1, slide 15)

The old way (still works today on Mac and Linux): `ifconfig -a`. The new way using “iproute2” tools on Linux: `ip link`, `ip addr`. What else do you see in these outputs? (MAC addresses, MTU, interface state, IPv6 addresses, the prefix length...)

2 Module 7.2 — Inside a Router: Route Lookup and Longest Prefix Matching

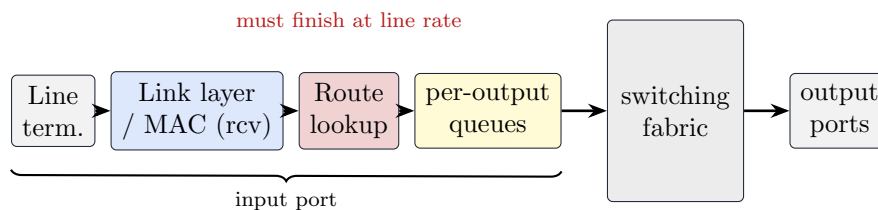
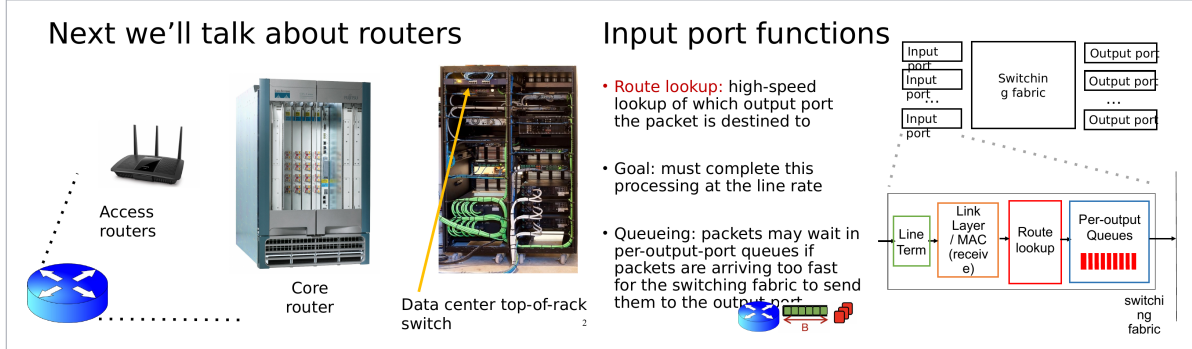
2.1 Kinds of routers

What the slide says (W7 L2, slide 2)

Access routers (e.g. your home WiFi router), core routers (huge chassis in ISP backbones), data-centre top-of-rack switches.

2.2 What's inside a router?

From the lecture slides: W7 L2, slides 2 and 4 — kinds of routers; input port functions



What the slide says (W7 L2, slide 4 — input port functions)

Route lookup: high-speed lookup of which output port the packet is destined to. Goal: must complete this processing **at the line rate**. **Queueing:** packets may wait in per-output-port queues if packets are arriving too fast for the switching fabric to send them to the output port.

Formula: What “at line rate” means

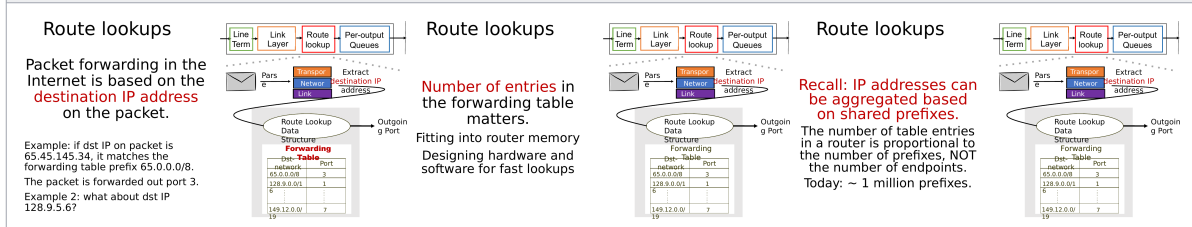
A 100 Gbps port receiving minimum-size (64-byte) packets gets one every

$$t = \frac{64 \times 8}{100 \times 10^9} = 5.12 \times 10^{-9} \text{ s} \approx 5 \text{ ns.}$$

The route lookup for each packet must finish in about 5 ns — a few memory accesses at most. That's why lookup data structures matter.

2.3 Route lookups

From the lecture slides: W7 L2, slides 5–7 — forwarding on destination IP; table size matters; entries scale with prefixes



What the slide says (W7 L2, slide 5)

Packet forwarding in the Internet is based on the **destination IP address** on the packet. The router parses the packet (link, network, transport headers), extracts the destination IP address, and looks it up in the forwarding table (a route-lookup data structure) to get the outgoing

port. Example: if the dst IP on the packet is 65.45.145.34, it matches the forwarding-table prefix 65.0.0.0/8; the packet is forwarded out port 3. Example 2: what about dst IP 128.9.5.6?

Dst network (prefix)	Port	Query	Result
65.0.0.0/8	3	65.45.145.34	matches /8 → port 3
128.9.0.0/16	1	128.9.5.6	matches 128.9.0.0/16 → port 1
149.12.0.0/19	7	149.12.40.1	/19 covers 149.12.0.0–149.12.31.255 → no match

What the slide says (W7 L2, slides 6–7)

The **number of entries** in the forwarding table matters: fitting into router memory; designing hardware and software for fast lookups. Recall: IP addresses can be aggregated based on shared prefixes. The number of table entries in a router is proportional to the **number of prefixes**, **NOT the number of endpoints**. Today: **~ 1 million prefixes**.

Intuition

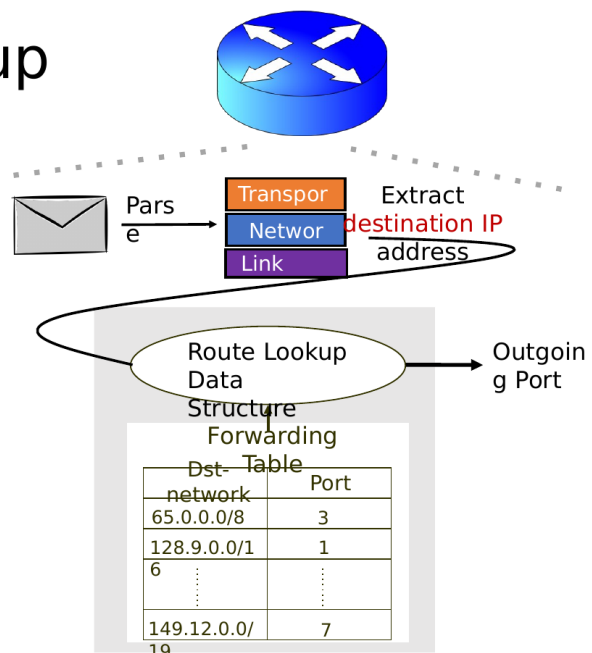
2^{32} individual entries (one per address) would need 4 billion rows — impossible at 5 ns per lookup. Aggregating into $\sim 10^6$ prefixes makes it feasible, but still demands clever structures (tries, TCAM hardware) because a match isn't a simple equality test.

2.4 Review, and IP block reallocation

From the lecture slides: W7 L2, slide 8 — a table lookup matches a packet against a prefix; prefixes are allocated and can be reallocated

Review: Route lookup

- Table lookup matches a packet against an IP **prefix**
 - Ex: 65.12.45.2 matches 65.0.0.0/8
- Prefixes are allocated to organizations by Internet registries
- But organizations can reallocate a subset of their IP address allocation to other orgs



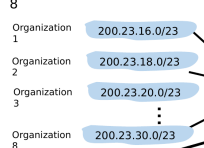
What the slide says (W7 L2, slide 8)

Table lookup matches a packet against an IP prefix (e.g. 65.12.45.2 matches 65.0.0.0/8). Prefixes are allocated to organisations by **Internet registries**. But organisations can **reallocate a subset** of their IP address allocation to other organisations.

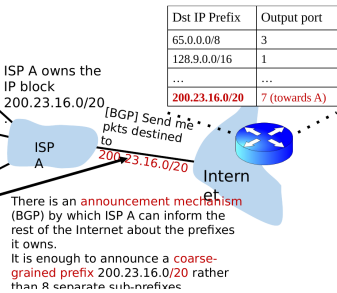
From the lecture slides: W7 L2, slides 9–10 — ISP A owns 200.23.16.0/20 and gives /23s to organisations 1–8; route aggregation

Example of IP block reallocation

Suppose ISP A reallocates a part of its IP block to orgs 1...8



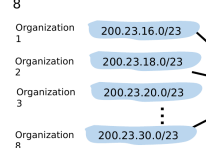
Route Aggregation
Save forwarding table



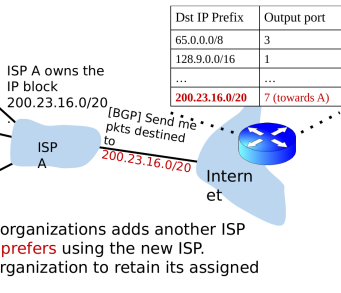
There is an announcement mechanism (BGP) by which ISP A can inform the rest of the Internet about the prefixes it owns. It is enough to announce a coarse-grained prefix 200.23.16.0/20 rather than 8 separate sub-prefixes

Example of IP block reallocation

Suppose ISP A reallocates a part of its IP block to orgs 1...8



Now suppose one of these organizations adds another ISP for its Internet service and prefers using the new ISP. Note: it's possible for the organization to retain its assigned IP block



What the slide says (W7 L2, slides 9–10)

Suppose ISP A reallocates a part of its IP block to organisations 1...8: 200.23.16.0/23, 200.23.18.0/23, 200.23.20.0/23, ..., 200.23.30.0/23. ISP A owns the block 200.23.16.0/20. There is an announcement mechanism (**BGP**) by which ISP A can inform the rest of the Internet about the prefixes it owns: “send me packets destined to 200.23.16.0/20”. It is enough to announce the coarse-grained prefix 200.23.16.0/20 rather than 8 separate sub-prefixes: **route aggregation** saves forwarding-table space. Now suppose one of these organisations adds another ISP for its Internet service and prefers using the new ISP. Note: it's possible for the organisation to retain its assigned IP block.

200.23.16.0/20 (ISP A's block, 4096 addresses)

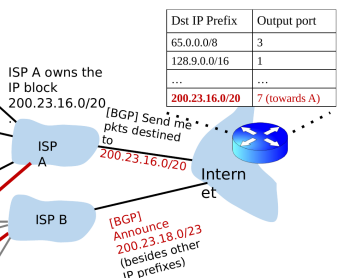
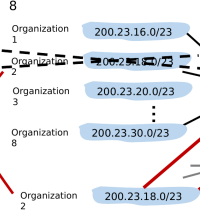
200.23.16.0/23	200.23.18.0/23	200.23.20.0/23	200.23.22.0/23	200.23.24.0/23	200.23.26.0/23	200.23.28.0/23	200.23.30.0/23
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------

Org 2 moves to ISP B

From the lecture slides: W7 L2, slides 11–12 — Organisation 2 connects via ISP B, which announces 200.23.18.0/23; both entries now in the table

Example of IP block reallocation

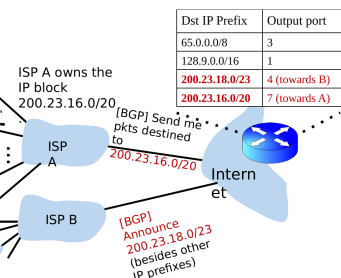
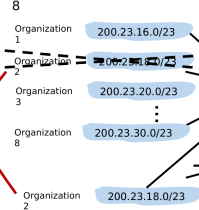
Suppose ISP A reallocates a part of its IP block to orgs 1...8



ISP B [BGP] Announce 200.23.18.0/23 (besides other IP prefixes)

Example of IP block reallocation

Suppose ISP A reallocates a part of its IP block to orgs 1...8



ISP B [BGP] Announce 200.23.18.0/23 (besides other IP prefixes)

2.5 A closer look at the forwarding table

Dst IP prefix	Output port
65.0.0.0/8	3
128.9.0.0/16	1
200.23.18.0/23	4 (towards B)
200.23.16.0/20	7 (towards A)

What the slide says (W7 L2, slide 13)

200.23.18.0/23 is **inside** 200.23.16.0/20. A packet with destination 200.23.18.xx is in **both** prefixes, i.e. both entries match. Q: how should the router choose to forward the packet? The org prefers B, so it should choose B.

2.6 Longest Prefix Matching (LPM)

From the lecture slides: W7 L2, slides 13–14 — overlapping prefixes; use the longest (most specific) match

A closer look at the forwarding table

- 200.23.18.0/23 is **inside** 200.23.16.0/20
- A packet with destination IP address 200.23.18.xx is in **both prefixes**
 - i.e., both entries match
- Q: How should the router choose to forward the packet?
 - The org prefers B, so should choose B

Dst IP Prefix	Output port
65.0.0.0/8	3
128.9.0.0/16	1
200.23.18.0/23	4 (towards B)
200.23.16.0/20	7 (towards A)



Longest Prefix Matching (LPM)

- Use the **longest** matching prefix, i.e., the most **specific** route, among all prefixes that match the packet.
- Policy borne out of the Internet's IP allocation model: prefixes and sub-prefixes are handed out
- **Internet routers use longest prefix matching.**
 - Very interesting algorithmic problems
 - Challenges in designing efficient software and hardware data structures

Dst IP Prefix	Output port
65.0.0.0/8	3
128.9.0.0/16	1
200.23.18.0/23	4 (towards B)
200.23.16.0/20	7 (towards A)



Definition: Longest prefix matching

Among all forwarding-table prefixes that match the destination address, use the **longest** one — the most **specific** route.

What the slide says (W7 L2, slides 14–15)

A policy borne out of the Internet's IP allocation model: prefixes and sub-prefixes are handed out. **Internet routers use longest prefix matching** on destination IP addresses of packets. Very interesting algorithmic problems; challenges in designing efficient software and hardware data structures.

Destination	Matching prefixes	LPM decision
200.23.19.5	/23 (200.23.18.0–19.255) and /20	/23 → port 4 (B)
200.23.20.1	/20 only	port 7 (A)
200.23.17.255	/20 only (17 is outside 18–19)	port 7 (A)
128.9.5.6	/16	port 1
10.0.0.1	none	default route / drop

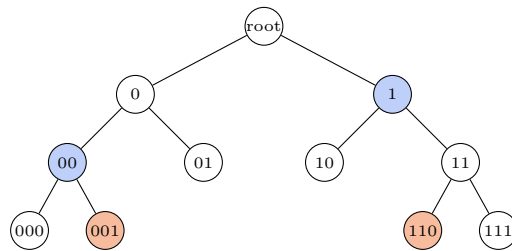
Intuition: Exceptions override the rule

The /20 is the general rule (“all of ISP A’s customers go via A”); the /23 is an exception (“except Org 2, which now goes via B”). A more specific prefix is always a more specific statement, so it should win. It also means ISP A never has to de-aggregate its /20: Org 2’s exception can live alongside it.

Beyond the slides: How routers do LPM fast

Binary trie: walk the destination’s bits from the top; remember the last node that marked a prefix; up to 32 steps. Compressed tries (Patricia) and multibit tries cut the steps. **TCAM** hardware compares against all prefixes in parallel in one clock cycle, with prefixes sorted by length so the first hit is the longest. The default route 0.0.0.0/0 matches everything (length

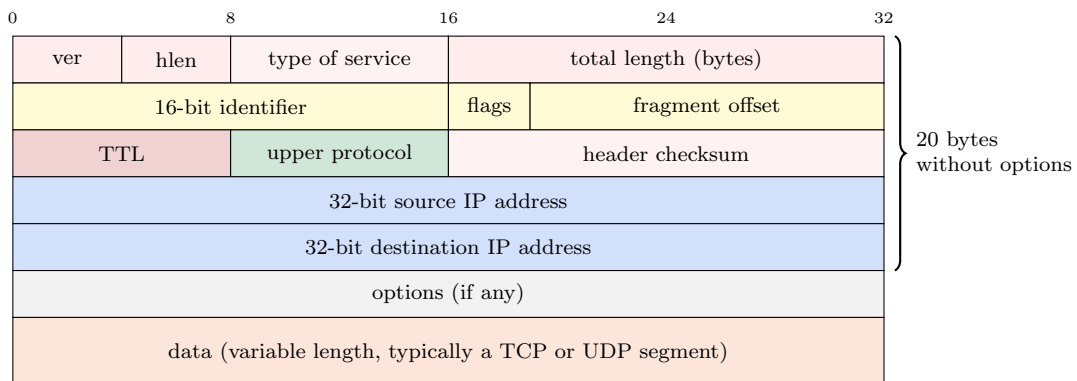
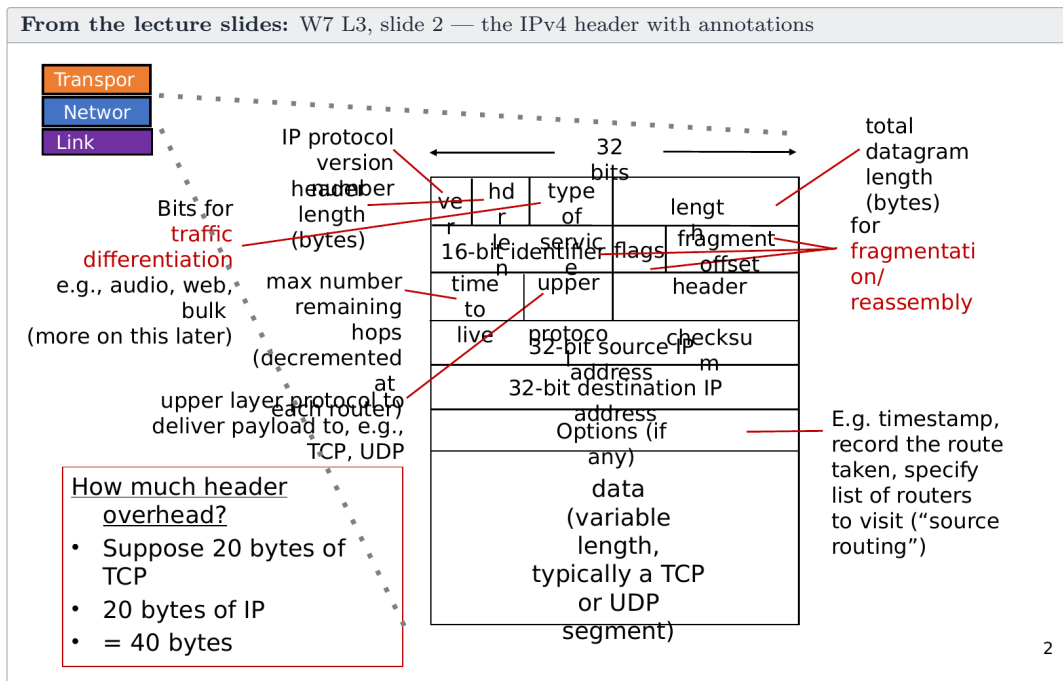
0), so it is used only when nothing more specific matches.



shaded nodes = prefixes in the table
lookup: follow bits, remember the
deepest shaded node passed $\Rightarrow$ LPM

3 Module 7.3 — The IPv4 Datagram and ICMP

3.1 IPv4 datagram format



Field	Meaning (from the slide annotations)
Version	IP protocol version number (4)
Header length	in 32-bit words (5 = 20 bytes; more with options)
Type of service	bits for traffic differentiation, e.g. audio vs web vs bulk (more later)
Total length	datagram length in bytes, header + data (max 65,535)
Identifier, flags, fragment offset	for fragmentation / reassembly
TTL (time to live)	max number of remaining hops, decremented at each router
Upper-layer protocol	which protocol to deliver the payload to, e.g. TCP (6), UDP (17), ICMP (1)
Header checksum	Internet checksum over the header only (recomputed at each hop, since TTL changes)
Source / destination IP	32-bit addresses
Options	e.g. timestamp, record the route taken, specify a list of routers to visit ("source routing")

Formula: Header overhead (slide)

TCP header 20 bytes + IP header 20 bytes = **40 bytes** per packet. Overhead fraction for payload P :

$$\frac{40}{40 + P} \quad [\text{dimensionless}]$$

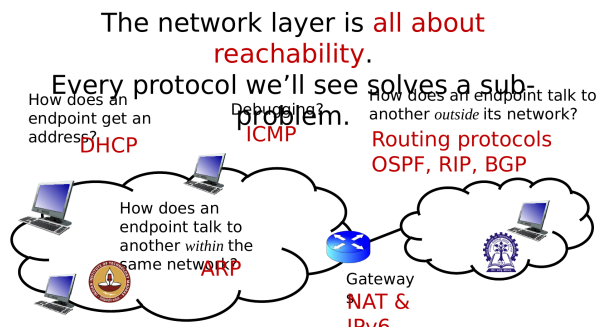
1460-byte payload: $40/1500 = 2.7\%$. 40-byte payload (e.g. a keystroke over SSH): $40/80 = 50\%$.

3.2 Various support protocols

From the lecture slides: W7 L3, slides 3–4 — support protocols; the network layer is all about reachability

Various Protocols

- Discuss **support protocols and mechanisms** for the network layer
 - Protocols: DHCP, **ICMP**, ARP, IPv6, ...
 - Mechanisms: **NAT**
- Some of these protocols use an IP header underneath their own header (**ICMP**) or replace the IP header with their own (**ARP**)
 - But these shouldn't be construed as transport/network protocols
 - They are fundamental to supporting IP/network layer functionality
 - More appropriately discussed as support protocols for the network layer



What the slide says (W7 L3, slide 3)

Discuss support protocols and mechanisms for the network layer — protocols: DHCP, ICMP, ARP, IPv6, ...; mechanisms: NAT. Some of these protocols use an IP header underneath their own header (**ICMP**) or replace the IP header with their own (**ARP**). But these shouldn't be construed as transport/network protocols: they are fundamental to supporting IP/network-layer functionality, more appropriately discussed as **support protocols** for the network layer.

What the slide says (W7 L3, slide 4 — the network layer is all about reachability)

Every protocol we'll see solves a sub-problem:

- How does an endpoint get an address? $\Rightarrow$ **DHCP**
- Debugging? $\Rightarrow$ **ICMP**
- How does an endpoint talk to another within the same network? $\Rightarrow$ **ARP**
- How does an endpoint talk to another outside its network? $\Rightarrow$ **Routing protocols**

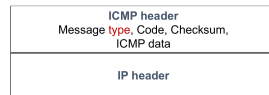
(OSPF, RIP, BGP), gateways, **NAT & IPv6**

3.3 Internet Control Message Protocol (ICMP)

From the lecture slides: W7 L3, slides 6–8 — ICMP: purpose, informal format (ICMP header over IP header), uses

Internet Control Message Protocol ICMP message format (informal)

- A protocol for **troubleshooting** and diagnostics
- Works over IP: **unreliable delivery** of packets
- Some functions of ICMP:
 - Determine reachability and network errors
 - Specify that packets have been in the network for too long



https://en.wikipedia.org/wiki/Internet_Control_Message_Protocol#Control_messages

Specific uses of ICMP

- **Echo request reply**
 - Check remotely if an endpoint is alive and connected
 - Without running an app remotely or controlling that endpoint
- **An unreachable destination**
 - Invalid address and/or port
- **Knowing if packet's IP time-to-live expired**
 - Example, due to routing loops
- Look at two tools built using ICMP: **ping** and **traceroute**

What the slide says (W7 L3, slides 6–7)

A protocol for **troubleshooting and diagnostics**. Works over IP: unreliable delivery of packets. Some functions: determine reachability and network errors; specify that packets have been in the network for too long. Informal format: ICMP header (message **type**, **code**, checksum, ICMP data) carried inside an IP packet (IP header underneath).

Type	Code	Meaning
0	0	echo reply (ping)
3	0 / 1 / 3	destination unreachable: network / host / port unreachable
8	0	echo request (ping)
11	0	time exceeded: TTL expired in transit (traceroute)

What the slide says (W7 L3, slide 8 — specific uses of ICMP)

Echo request/reply: check remotely if an endpoint is alive and connected, without running an app remotely or controlling that endpoint. **An unreachable destination:** invalid address and/or port. **Knowing if a packet's IP TTL expired:** e.g. due to routing loops. Two tools built using ICMP: **ping** and **traceroute**.

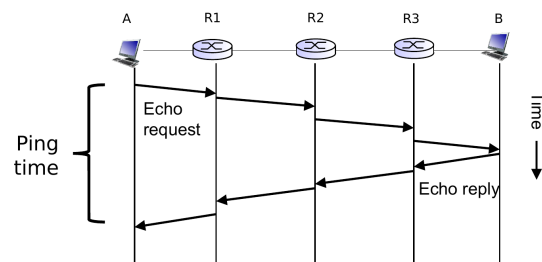
3.4 Ping

From the lecture slides: W7 L3, slides 9–10 — ping uses echo request (8,0) and echo reply (0,0); ping time = RTT

Ping

- Uses ICMP echo request (type=8, code=0) and reply (type=0, code=0)
- Source sends ICMP **echo request** message to dst address
- Destination network stack replies with an ICMP **echo reply** message
- Source can calculate round trip time (RTT) of packets
- If no echo reply comes back, then the destination is **unreachable**
- Don't need to have a server program running on the other side
 - In general, the remote endpoint can be completely outside your control

Ping



What the slide says (W7 L3, slide 9)

Uses ICMP echo request (type=8, code=0) and reply (type=0, code=0). The source sends an ICMP echo request message to the dst address; the destination network stack replies with an echo reply. The source can calculate the round-trip time (RTT). If no echo reply comes back, the destination is unreachable. No need for a server program on the other side — in general, the remote endpoint can be completely outside your control.

Intuition

The reply is generated by the remote *operating system's network stack*, not an application — which is why you can ping any machine that hasn't disabled it. (Many firewalls do block ICMP, so “no reply” can also mean “filtered”, not only “down”.) This is the tool behind Week 1's IIT Madras → IIT Delhi experiment.

3.5 Traceroute

From the lecture slides: W7 L3, slides 11–12 — a clever use of TTL; progressively increasing TTL

Traceroute

- A tool that can record the router-level path taken by packets
- A clever use of the IP **time-to-live** (TTL) field
- In general, when a router receives an IP packet, it decrements the TTL field on the packet
 - A failsafe mechanism to ensure packets don't keep taking up network resources for too long
- If a router receives a packet with TTL=0, it sends an **ICMP time exceeded** message (type=11, code=0) to the source endpoint

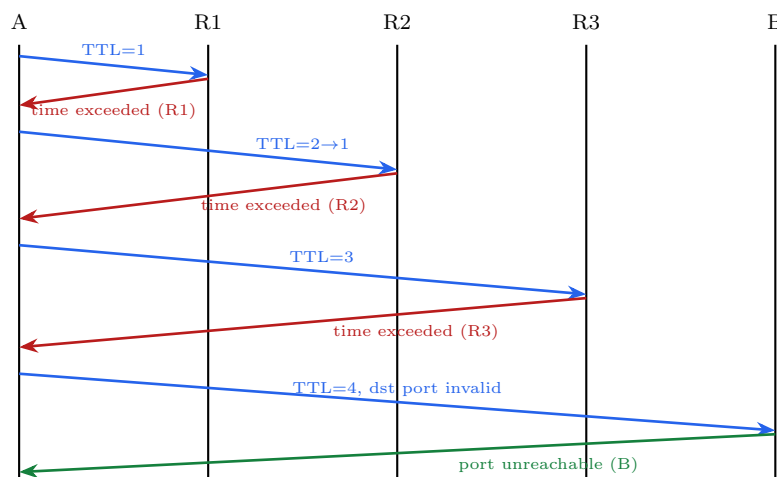
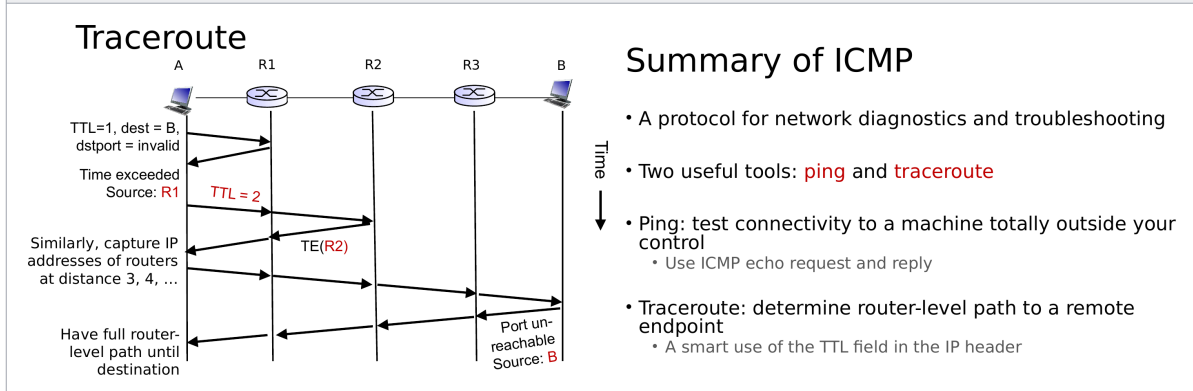
Traceroute

- Traceroute sends multiple packets to a destination endpoint
- But it **progressively increases the TTL** on those packets: 1, 2, ...
- Every time a time exceeded message is received, record the router's IP address
- Process repeated until the destination endpoint is reached
- If the packet reaches the destination endpoint (i.e.: TTL is high enough), then the endpoint sends a **port unreachable** message (type=3, code=3)

What the slide says (W7 L3, slides 11–12)

A tool that records the **router-level path** taken by packets — a clever use of the IP **time-to-live (TTL)** field. When a router receives an IP packet, it decrements the TTL (a failsafe to ensure packets don't keep taking up network resources for too long). If a router's decrement leaves TTL = 0, it sends an **ICMP time exceeded** message (type=11, code=0) to the source. Traceroute sends multiple packets to a destination endpoint, progressively increasing the TTL: 1, 2, Every time a time-exceeded message is received, record the router's IP address. Repeat until the destination is reached. When the packet reaches the destination (TTL high enough), the endpoint sends a **port unreachable** message (type=3, code=3) — because traceroute deliberately uses an invalid destination port.

From the lecture slides: W7 L3, slides 13–14 — TTL=1 expires at R1, TTL=2 at R2, ...; the destination replies port unreachable; ICMP summary



Formula: How many probes?

For a destination h router hops away, traceroute needs TTL values $1, 2, \dots, h + 1$: h time-exceeded replies (one per router) plus one port-unreachable from the destination. With k probes per TTL (the usual tool sends 3), total probes = $k(h + 1)$.

Common pitfall: TTL is a hop count

The TTL counts *router hops*, not time, despite its name. Each router decrements by 1; the router that drives it to 0 discards the packet and sends the time-exceeded message. Lines with * * * in real traceroute output are routers that don't send ICMP replies.

3.6 Summary of ICMP

Summary: ICMP (slide)

A protocol for network diagnostics and troubleshooting. Two useful tools: **ping** (test connectivity to a machine totally outside your control using ICMP echo request and reply) and **traceroute** (determine the router-level path to a remote endpoint — a smart use of the TTL field in the IP header).

4 Formula Sheet

IPv4 address	32 bits; dotted quad $a.b.c.d = a 2^{24} + b 2^{16} + c 2^8 + d$
Addresses in a /x	2^{32-x} (usable hosts $2^{32-x} - 2$)
Netmask for /x	x ones then zeros; partial byte $\in \{128, 192, 224, 240, 248, 252, 254\}$
Network address	$A \& M$
Same network?	$(A \& M) = (B \& M)$
Block b/y inside a/x	$y \geq x$ and first x bits agree
Forwarding	longest prefix match on destination IP
Line-rate lookup budget	packet size / link rate (64 B at 100 Gbps ≈ 5 ns)
IP + TCP header	$20 + 20 = 40$ bytes
ICMP types	echo req 8/0, echo reply 0/0, time exceeded 11/0, port unreachable 3/3
Traceroute probes	TTL = $1 \dots h + 1$; total $k(h + 1)$

5 Numerical Problems with Worked Solutions

Problem 1. Dotted quad conversions

(a) Convert 10000000 11000011 00000001 01010000 to dotted quad. (b) Convert 192.168.1.10 to binary. (c) Convert 172.16.254.3 to a 32-bit integer.

Solution

(a) 128.195.1.80 (the slide's example: $11000011 = 128 + 64 + 2 + 1 = 195$, $01010000 = 64 + 16 = 80$). (b) 11000000 10101000 00000001 00001010. (c) $172 \cdot 2^{24} + 16 \cdot 2^{16} + 254 \cdot 2^8 + 3 = 2,885,681,152 + 1,048,576 + 65,024 + 3 = \mathbf{2,886,794,755}$.

Problem 2. Block sizes

How many addresses are in a /8, /16, /20, /23, /24, /26 and /30?

Solution

$2^{24} = 16,777,216$; $2^{16} = 65,536$; $2^{12} = 4096$; $2^9 = 512$; $2^8 = 256$; $2^6 = 64$; $2^2 = 4$ (a /30 is used for point-to-point links: 2 usable hosts).

Problem 3. Netmasks

Write the netmask for /19, /20, /23, /26 and /28. What prefix length is 255.255.252.0?

Solution

/19: 255.255.224.0; /20: 255.255.240.0; /23: 255.255.254.0; /26: 255.255.255.192; /28: 255.255.255.240. $252 = 11111100$ (6 ones) $\Rightarrow 16 + 6 = \mathbf{/22}$.

Problem 4. Same network? (slide example + variant)

(a) A = 165.230.82.52, B = 165.230.24.93, M = 255.255.128.0. (b) Same A, B with M = 255.255.192.0.

Solution

(a) Third bytes: $82 \& 128 = 0$, $24 \& 128 = 0 \Rightarrow$ both 165.230.0.0: **same network**. (b) $82 \& 192$: $82 = 01010010$, $192 = 11000000 \Rightarrow 01000000 = 64$. $24 = 00011000 \Rightarrow 0$. Networks 165.230.64.0 vs 165.230.0.0: **different networks**. A longer mask draws finer boundaries.

Problem 5. *CIDR delegation (slide)*

An ISP owns 200.8.0.0/16. A customer needs 64 addresses starting at 200.8.4.128. (a) Give the block in CIDR notation and its range. (b) Its netmask? (c) Is it inside the ISP's block? (d) Could the customer be given 200.8.4.100/26?

Solution

(a) $64 = 2^6 \Rightarrow$ 6 host bits $\Rightarrow$ **200.8.4.128/26**, range 200.8.4.128 – 200.8.4.191. (b) 255.255.255.192. (c) Yes: $26 \geq 16$ and the first 16 bits are 200.8. (d) No: 100 is not a multiple of 64, so it isn't a valid /26 start ($100 \& 192 = 64 \Rightarrow$ that /26 is really 200.8.4.64/26).

Problem 6. *Splitting a block*

ISP A owns 200.23.16.0/20 and gives each of 8 organisations an equal block. What prefix length and which blocks? How many addresses each?

Solution

$8 = 2^3$ blocks $\Rightarrow 20 + 3 =$ **/23**, 512 addresses each: 200.23.16.0/23, 200.23.18.0/23, 200.23.20.0/23, 200.23.22.0/23, 200.23.24.0/23, 200.23.26.0/23, 200.23.28.0/23, 200.23.30.0/23 (the slide's organisations 1–8).

Problem 7. *Longest prefix matching*

Forwarding table: 65.0.0.0/8 $\rightarrow$ 3; 128.9.0.0/16 $\rightarrow$ 1; 149.12.0.0/19 $\rightarrow$ 7; 200.23.18.0/23 $\rightarrow$ 4; 200.23.16.0/20 $\rightarrow$ 7; default $\rightarrow$ 0. Where do these go: 65.45.145.34, 128.9.5.6, 200.23.19.5, 200.23.20.1, 200.23.17.255, 149.12.40.1, 149.12.31.200?

Solution

65.45.145.34 $\rightarrow$ 3. 128.9.5.6 $\rightarrow$ 1. 200.23.19.5 matches /23 and /20 $\Rightarrow$ longest /23 $\rightarrow$ 4. 200.23.20.1: only /20 $\rightarrow$ 7. 200.23.17.255: /23 covers only 18.0–19.255 $\Rightarrow$ /20 $\rightarrow$ 7. 149.12.40.1: /19 covers 149.12.0.0–149.12.31.255; $40 > 31 \Rightarrow$ **default 0**. 149.12.31.200 $\rightarrow$ 7.

Problem 8. *Route aggregation savings*

An ISP has 32 customer /24s: 10.0.0.0/24 through 10.0.31.0/24. What single prefix covers them, and how many table entries does it save elsewhere?

Solution

$32 = 2^5$ consecutive /24s aligned at 0 $\Rightarrow$ **10.0.0.0/19** (third byte 0–31). Other routers need 1 entry instead of 32: saves **31** entries per router.

Problem 9. *Line-rate lookup*

A 10 Gbps port receives back-to-back 64-byte packets. How much time per lookup? At 1500-byte packets?

Solution

$512/10^{10} = \mathbf{51.2 \text{ ns}}$; $12,000/10^{10} = \mathbf{1.2 \text{ }\mu\text{s}}$. Routers are designed for the worst case (smallest packets).

Problem 10. *Header overhead*

Using the slide's 40-byte TCP/IP header: overhead for payloads of 1460 bytes, 512 bytes and 1 byte? Throughput of useful data on a 100 Mbps link with 1460-byte payloads (ignore link headers)?

Solution

$40/1500 = 2.67\%$; $40/552 = 7.2\%$; $40/41 = 97.6\%$. Goodput = $100 \times 1460/1500 = \mathbf{97.3 \text{ Mbps}}$.

Problem 11. *Traceroute*

The destination is 6 router hops away; traceroute sends 3 probes per TTL. (a) Which TTLs are used? (b) How many time-exceeded and how many port-unreachable messages come back? (c) Total probes?

Solution

(a) TTL = 1, ..., 7. (b) $6 \times 3 = \mathbf{18}$ time-exceeded (type 11) and $\mathbf{3}$ port-unreachable (type 3, code 3). (c) $3 \times 7 = \mathbf{21}$ probes.

Problem 12. *TTL and a routing loop*

A packet is sent with TTL 64 and enters a routing loop between two routers 1 ms apart. How long does it circulate and what message is generated?

Solution

Each hop decrements TTL; after 64 hops ($\approx 64 \text{ ms}$) the router that drives it to 0 drops it and sends **ICMP time exceeded** (11,0) to the source. Without TTL it would loop forever.

6 Multiple-Choice Questions with Explanations

Q1. Internet addresses primarily help:

- (a) Name endpoints permanently
- (b) Routers decide how to move a packet
- (c) Encrypt traffic
- (d) Assign port numbers

(b).

Q2. An IPv4 address is:

- (a) 48 bits
- (b) 32 bits
- (c) 64 bits
- (d) 128 bits

(b).

Q3. An IP address identifies:

- (a) The endpoint permanently
- (b) A network interface / point of attachment
- (c) The user
- (d) The application

(b). It changes when the endpoint moves.

Q4. 11000000 10101000 00000000 00000001 in dotted quad is:

- (a) 192.168.0.1
- (b) 192.168.1.0
- (c) 168.192.0.1
- (d) 192.160.0.1

(a).

Q5. In an IP address, the prefix corresponds to the:

- (a) Host
- (b) Network
- (c) Port
- (d) MAC

(b).

Q6. Prefixes help routing scale because:

- (a) They encrypt addresses
- (b) Routers store one entry per prefix, not per endpoint
- (c) They are shorter than MACs
- (d) They avoid ARP

(b). Like PIN codes.

Q7. CIDR stands for:

- (a) Classful Internet Domain Routing
- (b) Classless Inter-Domain Routing
- (c) Centralised IP Distribution Registry
- (d) Common Internet Datagram Relay

(b).

Q8. In classless addressing, the prefix length:

- (a) Is fixed at 8, 16 or 24
- (b) Can be any length, written a.b.c.d/x
- (c) Is encoded in the first bits
- (d) Is always 32

(b).

Q9. How many addresses are in a /26?

- (a) 26
- (b) 32
- (c) 64

(d) 128

(c). 2^6 .

Q10. The netmask for /23 is:

- (a) 255.255.255.0
- (b) 255.255.254.0
- (c) 255.255.252.0
- (d) 255.255.0.0

(b).

Q11. 200.8.4.128/26 is inside 200.8.0.0/16 because:

- (a) $26 > 16$ and the first 16 bits match
- (b) Both start with 200
- (c) /26 is smaller than /16
- (d) Any block is inside any other

(a).

Q12. Two addresses are on the same network if:

- (a) Their first bytes match
- (b) $A \& M$ equals $B \& M$
- (c) $A | M$ equals $B | M$
- (d) $A + B = M$

(b).

Q13. $165.230.82.52 \& 255.255.128.0 =$

- (a) 165.230.82.0
- (b) 165.230.0.0
- (c) 165.230.128.0
- (d) 165.0.0.0

(b).

Q14. A modern Linux command to show IP addresses is:

- (a) ip addr
- (b) ping
- (c) netstat -r
- (d) dig

(a).

Q15. Packet forwarding in the Internet is based on:

- (a) Source IP
- (b) Destination IP
- (c) Source port
- (d) MAC address of the sender

(b).

Q16. Route lookup must complete:

- (a) Within one RTT
- (b) At line rate
- (c) Within one second
- (d) Whenever convenient

(b).

Q17. Packets may queue in a router in:

- (a) The DNS cache
- (b) Per-output-port queues
- (c) The ARP table
- (d) The TTL field

(b).

Q18. The number of forwarding-table entries is proportional to:

- (a) Number of endpoints
- (b) Number of prefixes
- (c) Number of links
- (d) Number of ports

(b). About 1 million today.

Q19. Which destination does 65.0.0.0/8 match?

- (a) 66.1.1.1
- (b) 65.45.145.34
- (c) 128.65.0.1
- (d) 6.5.0.0

(b).

Q20. 128.9.5.6 in the slide's table goes out port:

- (a) 3
- (b) 1
- (c) 7
- (d) 4

(b). Matches 128.9.0.0/16.

Q21. ISP A announcing 200.23.16.0/20 instead of eight /23s is called:

- (a) Fragmentation
- (b) Route aggregation
- (c) NAT
- (d) Tunnelling

(b).

Q22. The protocol ISPs use to announce owned prefixes is:

- (a) OSPF
- (b) BGP

- (c) ARP
- (d) DHCP

(b).

Q23. A packet to 200.23.18.7 with entries 200.23.18.0/23 (port 4) and 200.23.16.0/20 (port 7) goes to:

- (a) Port 7
- (b) Port 4
- (c) Both
- (d) Dropped

(b). Longest prefix match.

Q24. Longest prefix matching chooses:

- (a) The first entry
- (b) The most specific matching prefix
- (c) The shortest prefix
- (d) A random match

(b).

Q25. The IPv4 header without options is:

- (a) 8 bytes
- (b) 20 bytes
- (c) 40 bytes
- (d) 60 bytes

(b).

Q26. The TTL field:

- (a) Counts seconds
- (b) Is decremented at each router
- (c) Is set by the receiver
- (d) Only exists in IPv6

(b).

Q27. The “upper-layer protocol” field tells the receiver:

- (a) The next router
- (b) Whether to deliver the payload to TCP, UDP, ICMP...
- (c) The TTL
- (d) The MTU

(b).

Q28. Which support protocol replaces the IP header with its own?

- (a) ICMP
- (b) ARP
- (c) TCP
- (d) UDP

(b). ICMP rides inside IP.

Q29. How does an endpoint get an address?

- (a) ARP
- (b) DHCP
- (c) ICMP
- (d) BGP

(b).

Q30. ICMP is used for:

- (a) Reliable file transfer
- (b) Troubleshooting and diagnostics
- (c) Routing table exchange
- (d) Address assignment

(b).

Q31. Ping uses ICMP messages of types:

- (a) 8 and 0
- (b) 11 and 3
- (c) 3 and 0
- (d) 1 and 2

(a). Echo request 8, echo reply 0.

Q32. Ping requires on the remote side:

- (a) A running server app
- (b) Nothing but a network stack that answers echo requests
- (c) SSH access
- (d) An open TCP port

(b).

Q33. When a router decrements TTL to 0 it sends:

- (a) Echo reply
- (b) Time exceeded (type 11)
- (c) Port unreachable
- (d) Nothing

(b).

Q34. Traceroute knows it has reached the destination when it receives:

- (a) Time exceeded
- (b) Port unreachable (type 3, code 3)
- (c) Echo reply always
- (d) A TCP RST

(b). It uses an invalid destination port.

Q35. Traceroute sends probes with TTL:

- (a) Always 64
- (b) 1, 2, 3, ... increasing
- (c) Decreasing
- (d) 0

(b).